

ServerBalancer

Оптимизация балансировки нагрузки методами
ТМО



Лобанов Р.Е. · Батраков Я.А.

Неравномерная загрузка серверов

- 3 сервера: $\mu = 100 / 80 / 60$ req/s
- Равное распределение $\rightarrow$ медленный сервер перегружен
- $W_2 = 1/(60-50) = 100$ мс при $\lambda_i = 50$ req/s
- Задача: минимизировать среднее время отклика W_{avg}

Математическая основа

- Пуассоновский поток λ , экспоненциальное обслуживание μ
- Время пребывания: $W = 1 / (\mu - \lambda)$
- Загрузка: $\rho = \lambda/\mu$, условие устойчивости $\rho < 1$
- Нелинейная зависимость: чем ближе ρ к 1 — тем больше W

ВРЕМЯ ПРЕБЫВАНИЯ

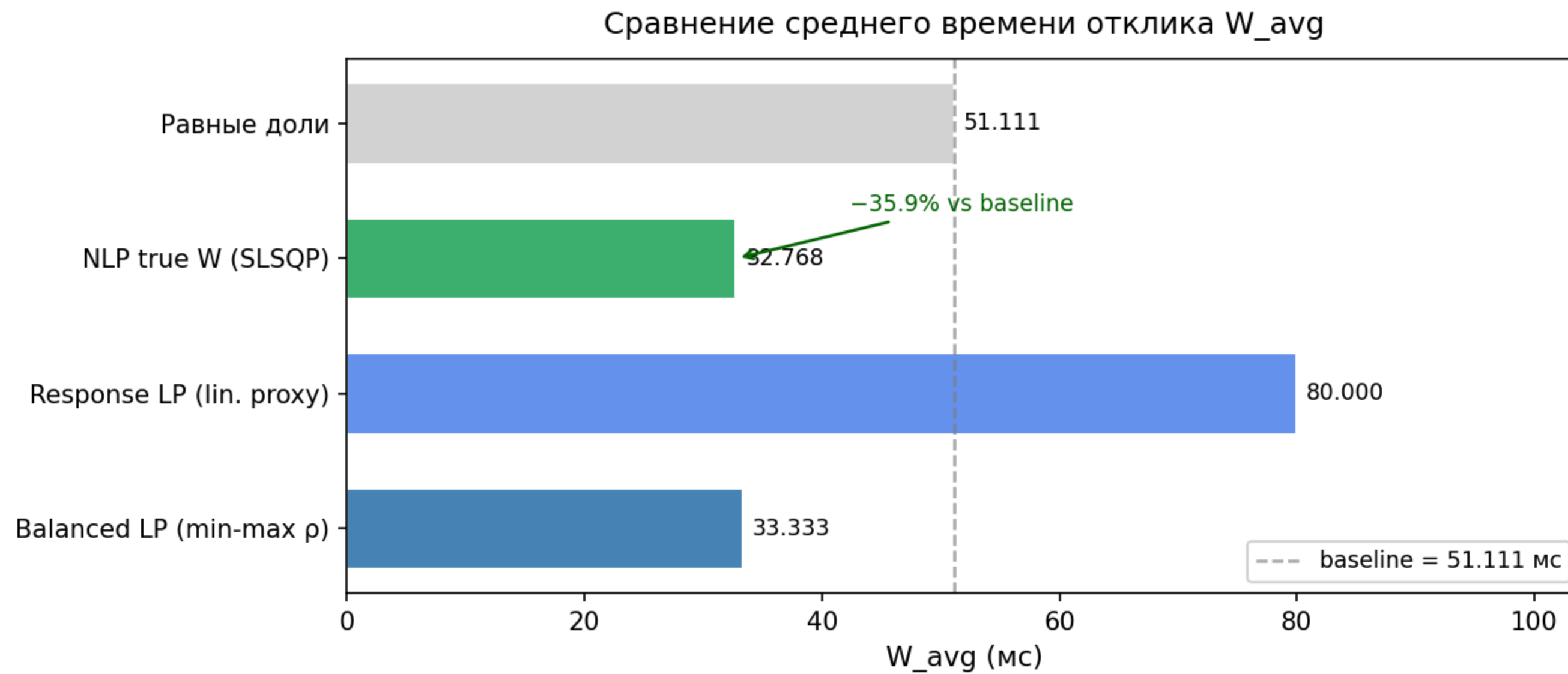
$$W = \frac{1}{\mu - \lambda}$$

Три метода оптимизации

Постановки задачи ($\sum x_i = 1, x_i \geq 0$)

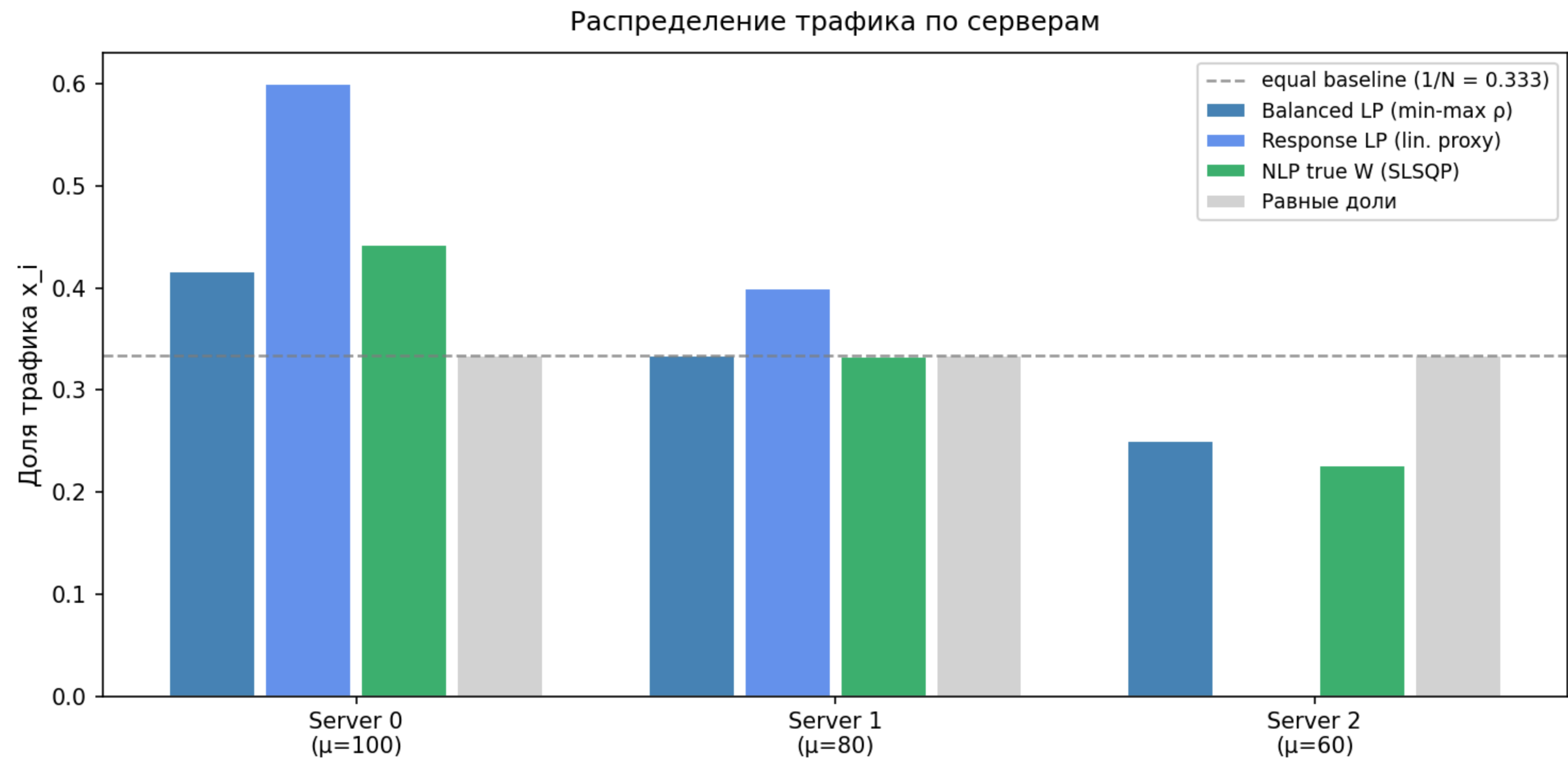
Метод	Целевая функция	Инструмент
LP balanced	$\min \max(\rho_i)$	PuLP / CBC
LP proxy	$\min \sum \lambda x_i / \mu_i^2$	PuLP / CBC
NLP SLSQP	$\min \sum x_i \cdot W_i$ (истинный W)	SciPy SLSQP

$\lambda = 150 \text{ req/s}$, $\mu = [100, 80, 60]$



NLP улучшает W_{avg} на ~36% по сравнению с равномерным распределением

Как меняются доли x_i



NLP отдаёт больше трафика быстрым серверам

Архитектура системы

СТЕК

- Python
- PuLP · SciPy
- SimPy
- FastAPI · Docker

МОДУЛИ

- `queue_math.py`
формулы M/M/1
- `lp_optimizer.py`
3 метода оптимизации
- `simulation.py`
SimPy валидация
- `api/app.py`
REST + дашборд

Итоги

- NLP SLSQP — наилучший результат: **$W_{avg} \approx 32.8 \text{ мс}$** vs 51 мс baseline
- LP применимо как быстрая аппроксимация без нелинейного решателя
- Модель M/M/1 подтверждена дискретно-событийной симуляцией
- Система готова к развёртыванию: FastAPI + Docker

–36% время отклика